

Creating Visualization

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [3]: df = pd.read_csv(r"Ice Cream Ratings.csv")
df = df.set_index('Date')
df
```

```
Out[3]:
```

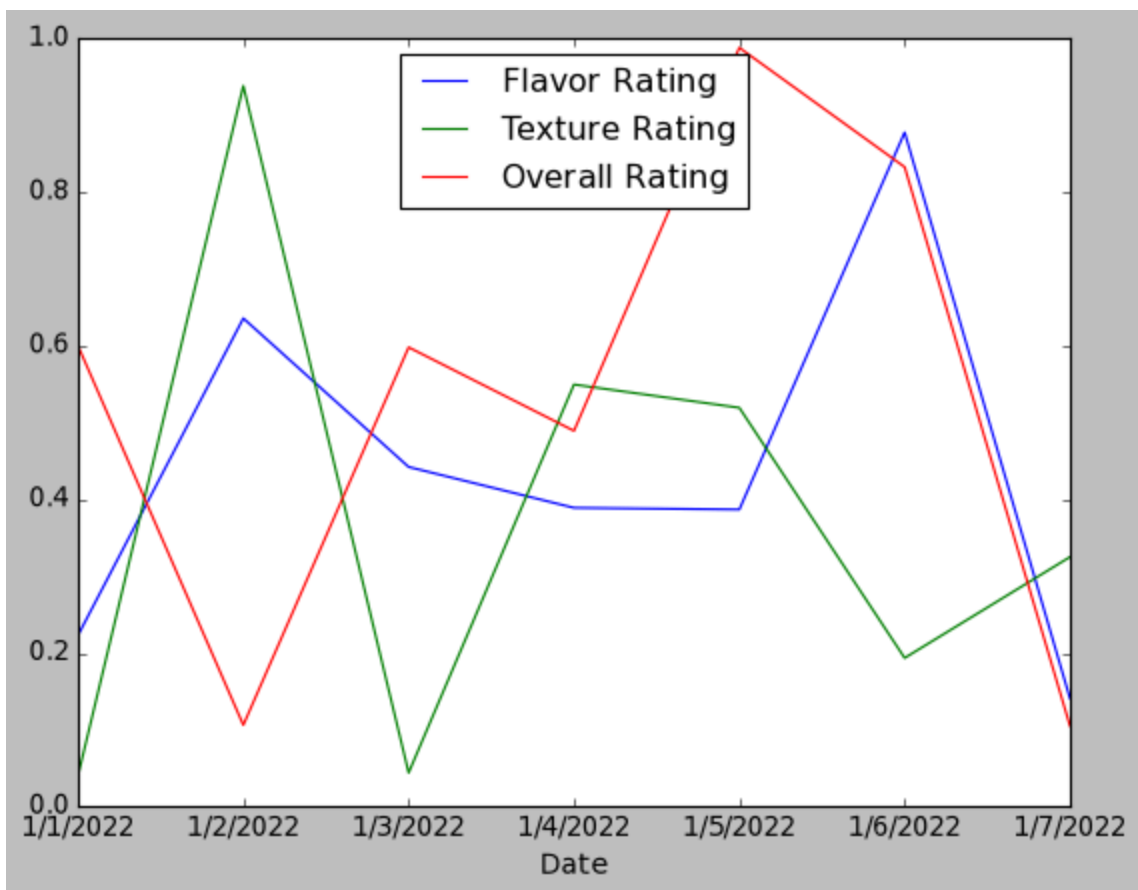
	Flavor Rating	Texture Rating	Overall Rating
Date			
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	0.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

```
In [8]: print(plt.style.available)

['Solarize_Light2', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight',
'ggplot', 'grayscale', 'petroff10', 'petroff6', 'petroff8', 'seaborn-v0_8', 'seaborn-
v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-pa
lette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-
v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster',
'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-white
grid', 'tableau-colorblind10']
```

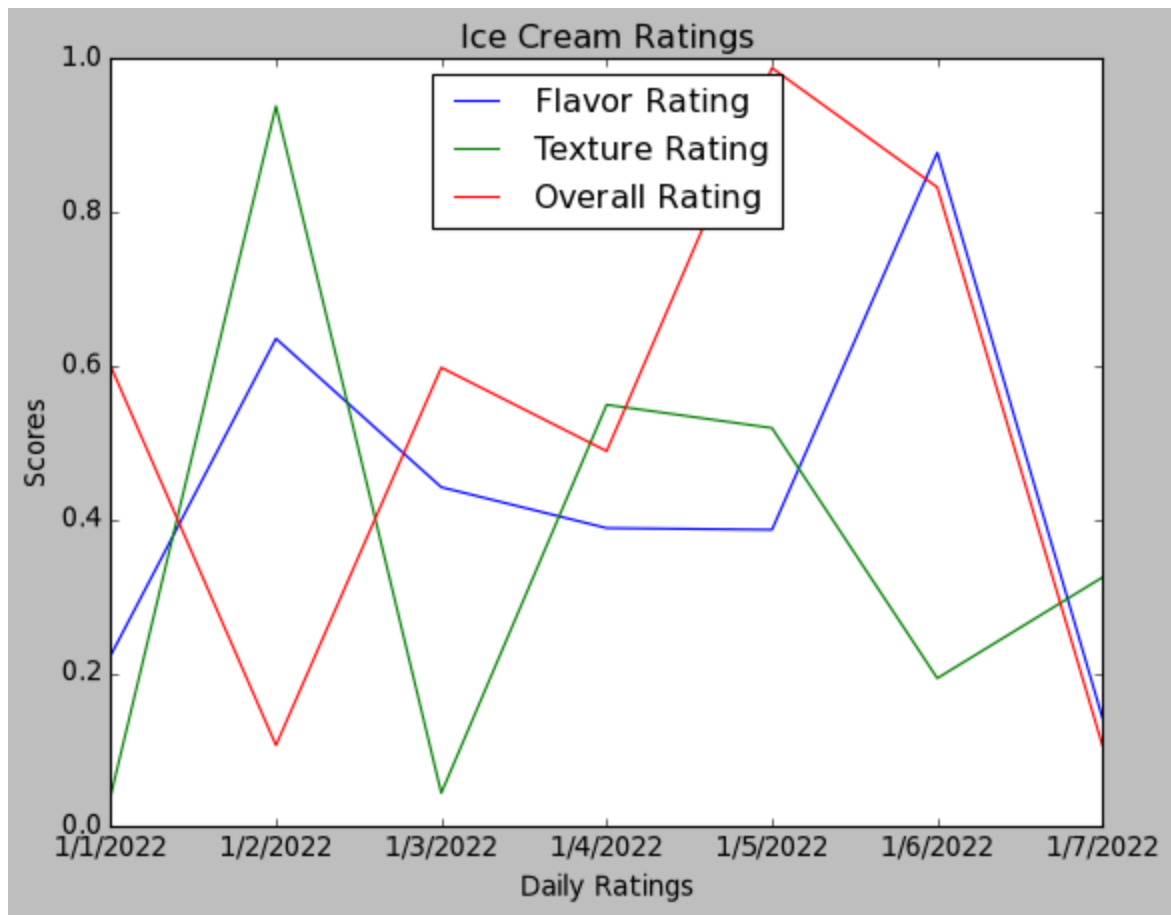
```
In [9]: plt.style.use('classic')
df.plot(kind = 'line')
```

```
Out[9]: <Axes: xlabel='Date'>
```



```
In [7]: df.plot(kind = 'line', title = 'Ice Cream Ratings', xlabel = 'Daily Ratings', ylabel = 'Scores')
```

```
Out[7]: <Axes: title={'center': 'Ice Cream Ratings'}, xlabel='Daily Ratings', ylabel='Scores'>
```



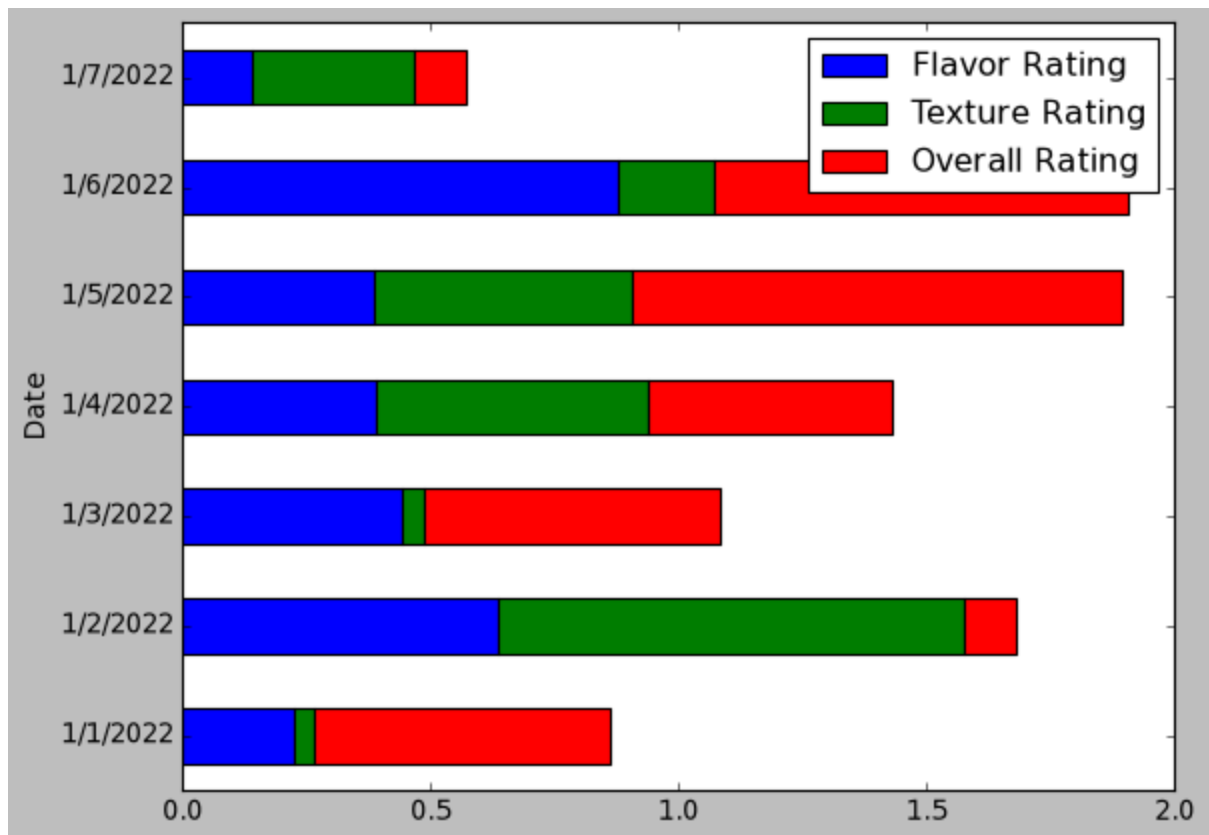
In [12]: df

Out[12]:

Date	Flavor Rating	Texture Rating	Overall Rating
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	0.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

In [10]: df.plot.barh(stacked = True)

Out[10]: <Axes: ylabel='Date'>



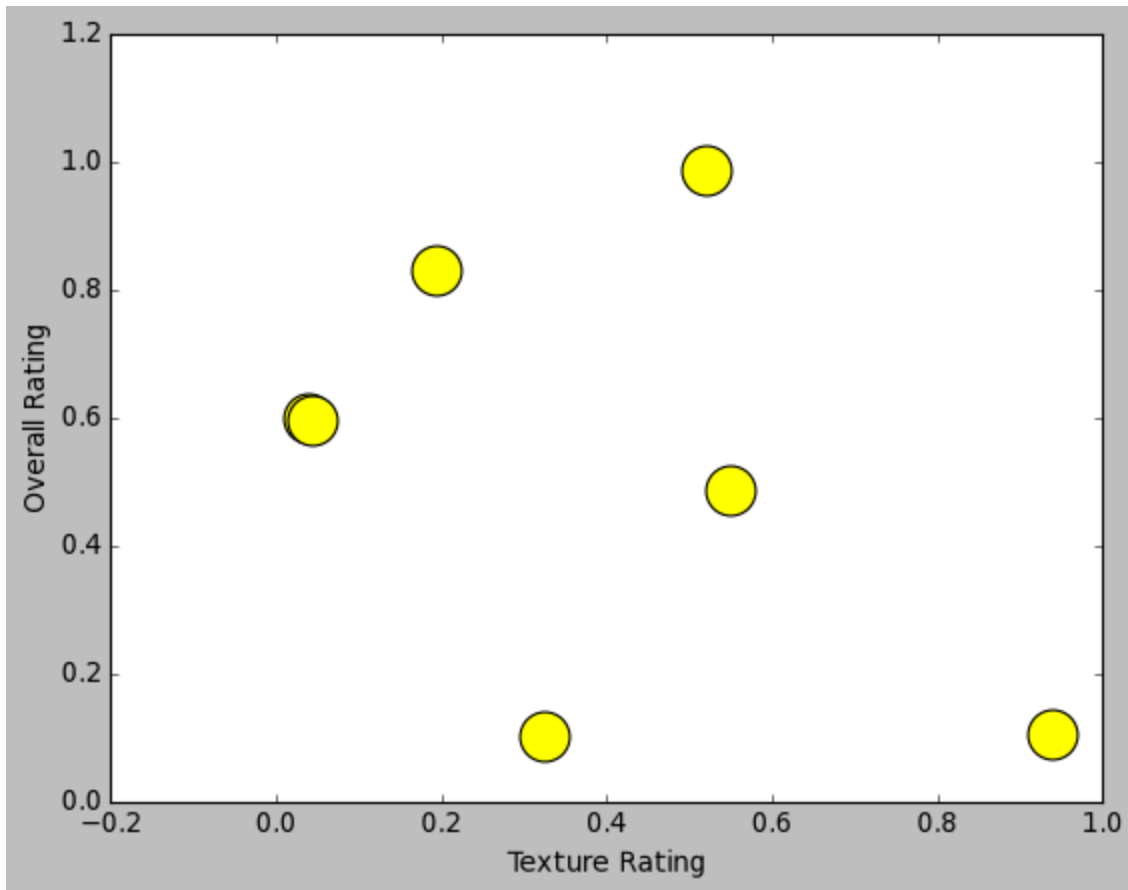
In [13]: df

Out[13]:

Date	Flavor Rating	Texture Rating	Overall Rating
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	1.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

In [11]: df.plot.scatter(x = 'Texture Rating', y = 'Overall Rating', s = 500, c = 'Yellow')

Out[11]: <Axes: xlabel='Texture Rating', ylabel='Overall Rating'>



`df.plot.scatter(...)`: Tells Pandas to create a scatter plot using your DataFrame (`df`), where each row in your dataset is represented as a single dot on the graph.

`x='Texture Rating'`: Places the Texture Rating values along the horizontal X-axis.

`y='Overall Rating'`: Places the Overall Rating values along the vertical Y-axis. Every dot's position tells you an item's specific combination of texture and overall score.

`s=500`: Sets the size (`s`) of every marker to 500. This is significantly larger than the default size, making the points bold and prominent (sometimes called a bubble chart style).

`c='Yellow'`: Paints every data point yellow.

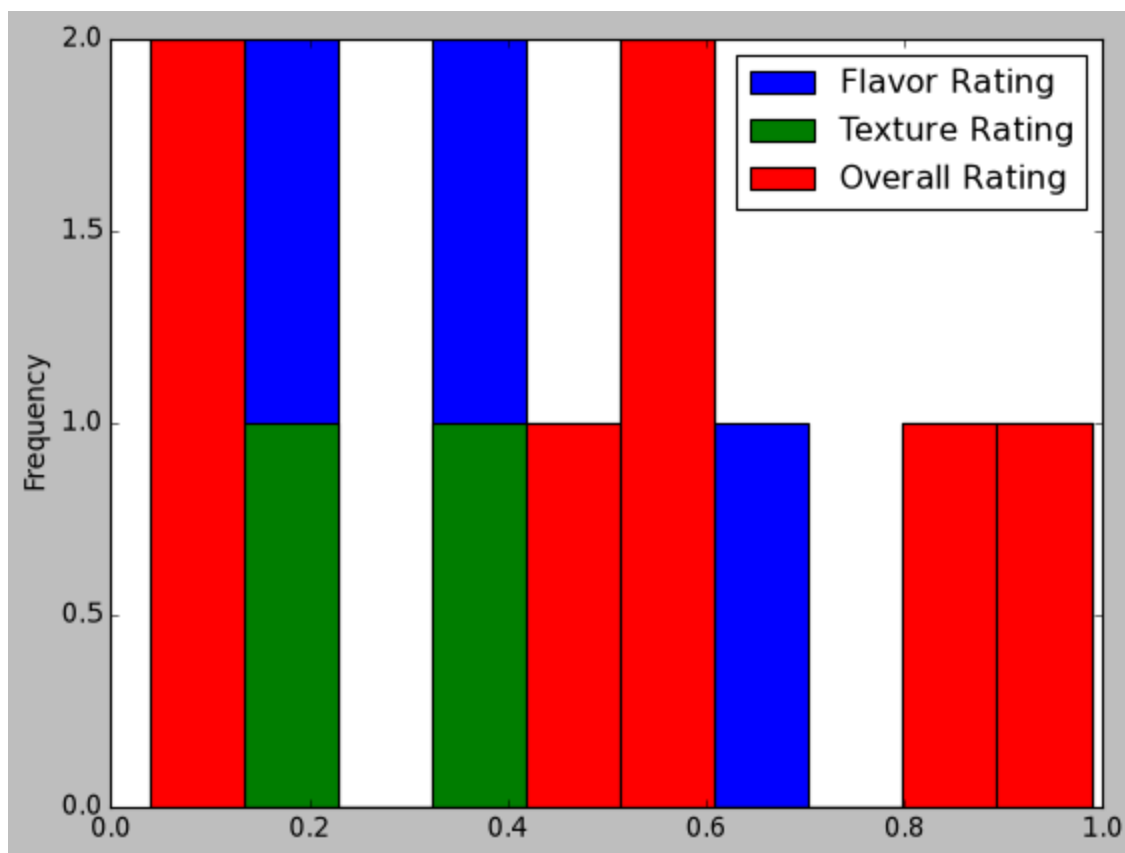
How to read the resulting plot: Correlation: Look at how the yellow dots trend. If the dots slope upward from bottom-left to top-right, it means items with better texture tend to have higher overall ratings. If they are scattered randomly or slope downward, texture has little or no positive impact on the overall score.

Clustering: Notice where the dots pack tightly together. That concentration highlights the most common "Texture vs. Overall Rating" profile across your dataset.

In [14]: `df`

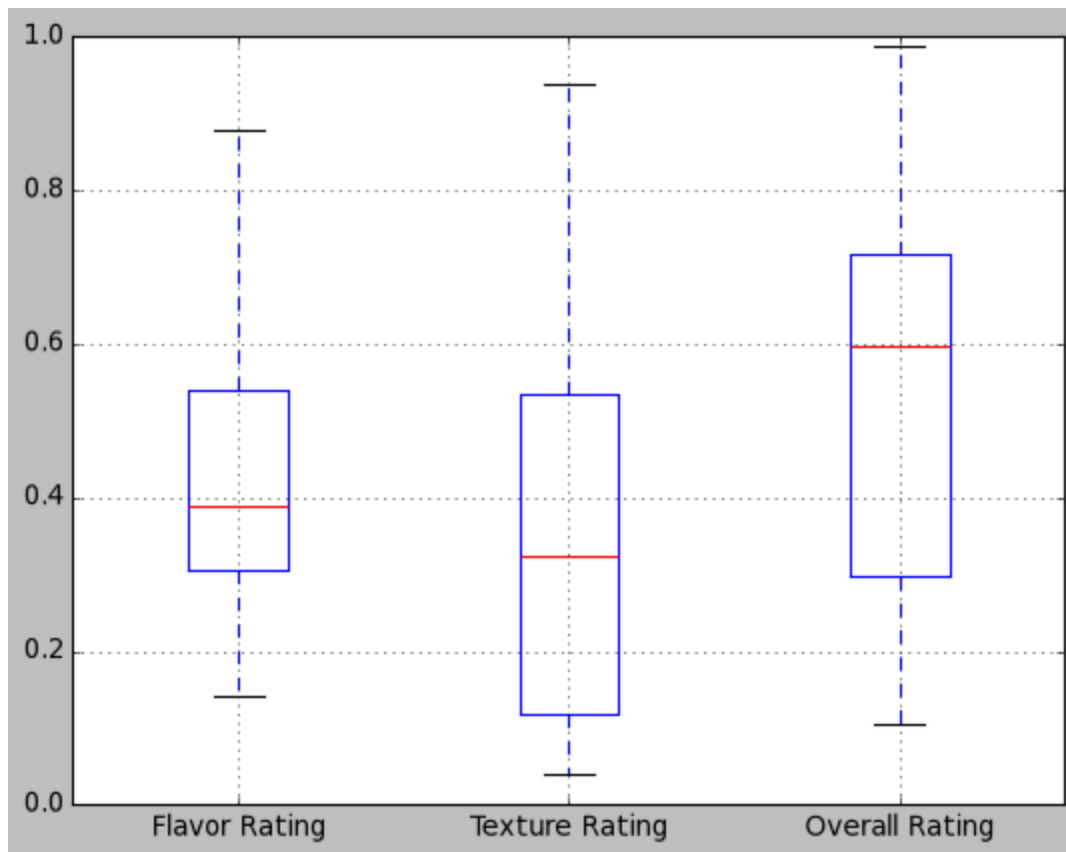
Out[14]:

	Flavor Rating	Texture Rating	Overall Rating
Date			
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	0.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

In [15]: `df.plot.hist(bins = 10)`Out[15]: `<Axes: ylabel='Frequency'>`In [16]: `df`

Out[16]:

	Flavor Rating	Texture Rating	Overall Rating
Date			
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	0.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

In [17]: `df.boxplot()`Out[17]: `<Axes: >`

A boxplot (box-and-whisker plot) provides a visual summary of the distribution, central tendency, variability, and potential outliers of numerical columns in your DataFrame.

Anatomy of the Box

Median (Q2): The horizontal line inside the box representing the 50th percentile (the exact middle of the data).

Bottom of the Box (Q1): The 25th percentile (lower quartile). 25% of the data falls below this value.

Top of the Box (Q3): The 75th percentile (upper quartile). 75% of the data falls below this value.

The Box (IQR): The height of the box represents the Interquartile Range ($IQR = Q3 - Q1$), containing the middle 50% of your data.

Whiskers and Outliers

Whiskers: Lines extending outward from the top and bottom of the box to indicate the spread of the rest of the dataset, typically stretching up to $1.5 \times \text{IQR}$ from the quartiles.

Outliers: Individual dots, circles, or asterisks plotted past the ends of the whiskers, flagging unusually high or low data points that warrant inspection.

Key Interpretations

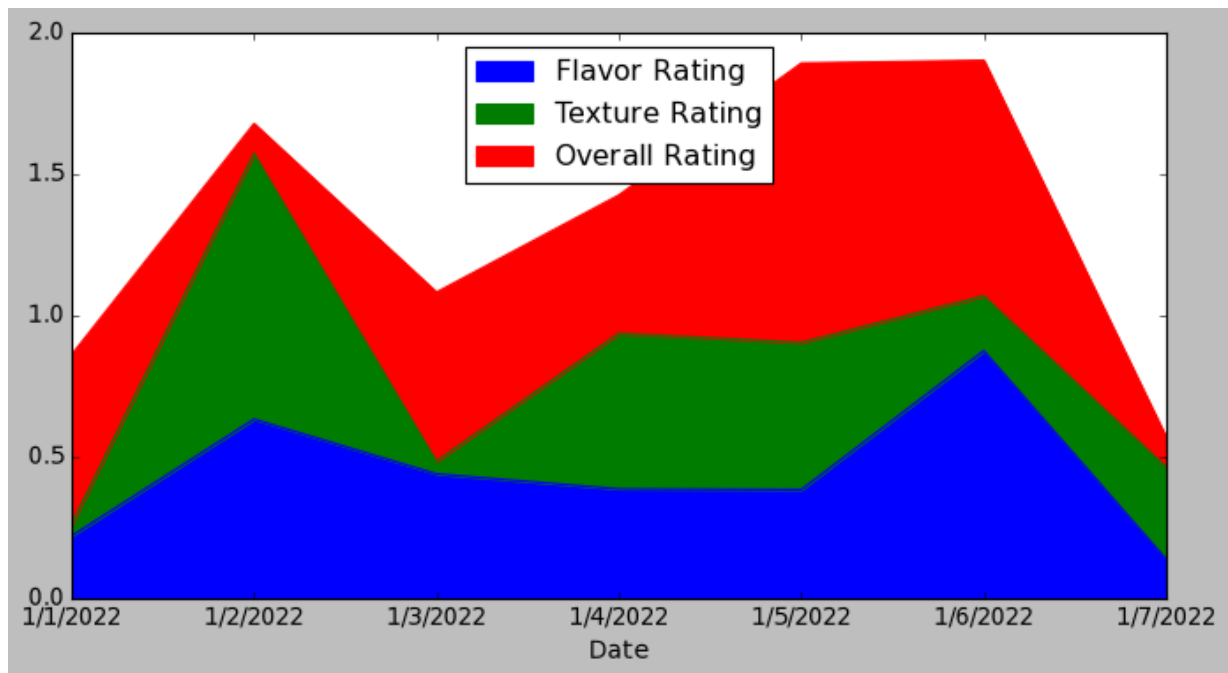
Skewness: If the median line sits closer to the bottom of the box, the data is right-skewed (positively skewed, with a long tail of high values). If it sits closer to the top, it is left-skewed.

Spread and Variability: A taller box and longer whiskers mean your data is widely dispersed. A short, compact box indicates values are tightly clustered around the median.

Side-by-Side Comparisons: When `df.boxplot()` generates multiple plots for different columns, you can instantly compare their medians and ranges to spot which columns have higher values or greater variance.

```
In [18]: df.plot.area(figsize = (10,5))
```

```
Out[18]: <Axes: xlabel='Date'>
```



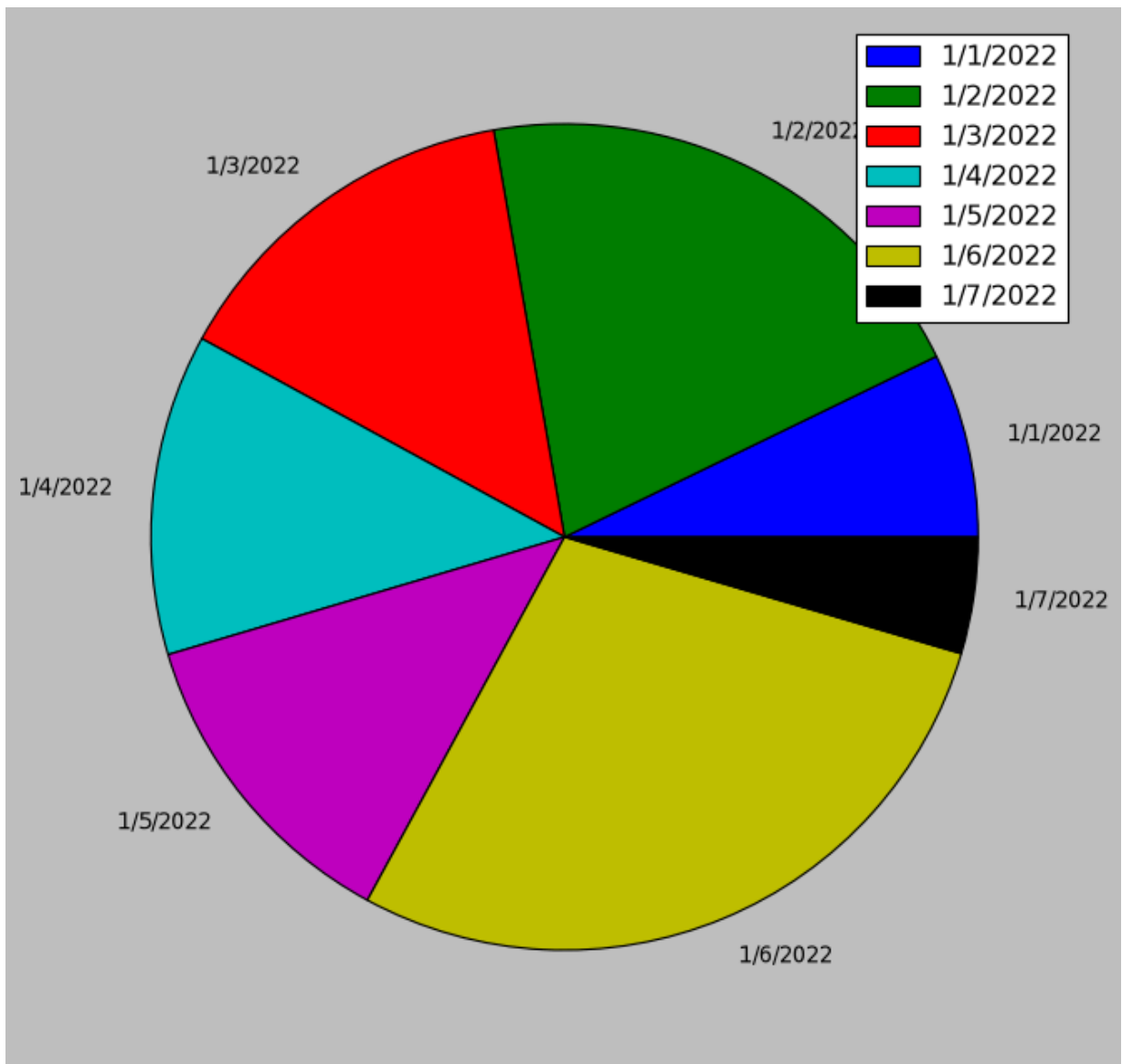
In [19]: df

Out[19]:

Date	Flavor Rating	Texture Rating	Overall Rating
1/1/2022	0.223090	0.040220	0.600129
1/2/2022	0.635886	0.938476	1.106264
1/3/2022	0.442323	0.044154	0.598112
1/4/2022	0.389128	0.549676	0.489353
1/5/2022	0.386887	0.519439	0.988280
1/6/2022	0.877984	0.193588	0.832827
1/7/2022	0.140995	0.325110	0.105147

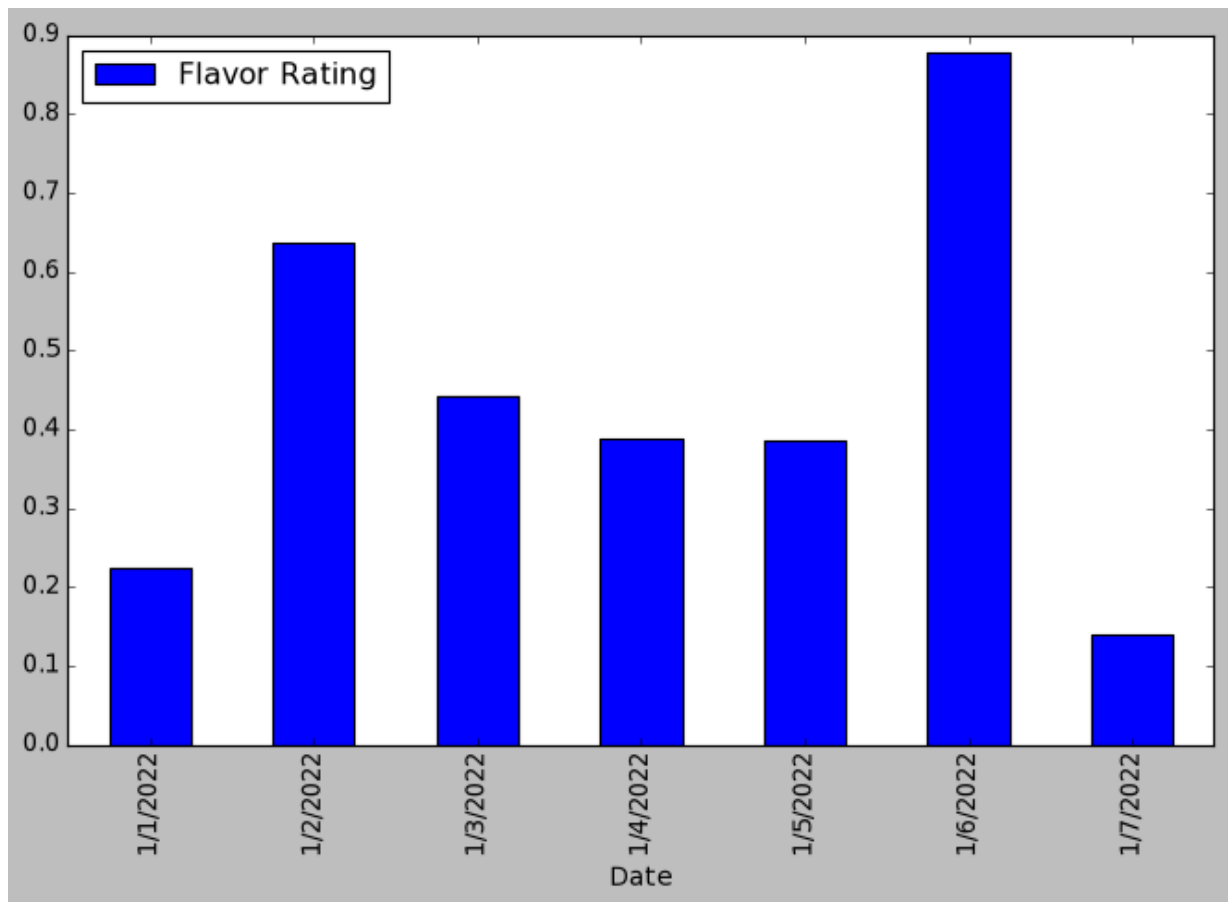
In [20]: df.plot.pie(y='Flavor Rating',figsize=(10,10))

Out[20]: <Axes: >



```
In [22]: df.plot.bar(y='Flavor Rating', figsize=(10, 6))
```

```
Out[22]: <Axes: xlabel='Date'>
```



In []: